



Sistema de Memoria y Razonamiento para tu Proyecto

Basándome en el documento y tu estructura actual, vamos a implementar una arquitectura STM + LTM + Episódica + CoT + FactChecker directamente en archivos Markdown (sin dependencias externas), compatible 100% con OpenCode.



Nueva Estructura Propuesta

text

```
mi-proyecto/
├── .ai/
│   ├── system.md
│   ├── rules.md
│   ├── context.md
│   └── agents.md
│
│   ├── memory/
│   │   ├── README.md
│   │   ├── STM.md
│   │   ├── LTM.md
│   │   ├── episodic.md
│   │   └── index.md
│   └── reasoning/
│       └── CoT.md
```

- #  Prompt base (sin cambios)
- #  Reglas globales (sin cambios)
- #  Contexto técnico (sin cambios)
- #  Agentes (sin cambios)

- #  **NUEVO:** Sistema de memoria
- # Cómo funciona el sistema de memoria
- # Memoria a Corto Plazo (sesión actual)
- # Memoria a Largo Plazo (conocimiento acumulado)
- # Memoria Episódica (historial de proyectos)
- # Índice maestro + búsqueda rápida

- #  **NUEVO:** Sistema de razonamiento
- # Chain of Thought - plantilla de razonamiento

```
├── factcheck.md # Verificación de hechos y confianza
├── patterns.md # Patrones de razonamiento aprendidos
├── logs/ # 📅 NUEVO: Registro de experiencia
│   ├── decisions.md # Decisiones tomadas + por qué
│   ├── errors.md # Errores cometidos + solución aplicada
│   └── learnings.md # Aprendizajes acumulados por proyecto
├── .vscode/
├── .env
├── .gitignore
├── opencode.json
└── src/
```



Contenido de cada archivo



```
.ai/memory/README.md
```

Markdown

Sistema de Memoria del Agente

Cómo funciona

La IA lee memoria en este orden ANTES de responder:

1. ****STM**** → ¿Qué estoy haciendo AHORA?
2. ****LTM**** → ¿Qué sé sobre este tema en general?
3. ****Episódica**** → ¿Hice algo similar antes? ¿Qué pasó?
4. ****Index**** → Búsqueda rápida por tags/proyecto

Cuando escribir en memoria

Evento	Dónde escribir
Inicio de sesión	STM.md
Decisión técnica importante	LTM.md + decisions.md
Completar una tarea	episodic.md + learnings.md
Error cometido y resuelto	errors.md + LTM.md
Fin de sesión	Mover STM relevante → LTM/episodic

Ciclo de vida de la memoria

STM (sesión) → [al cerrar] → LTM (si es conocimiento)
 → episodic (si es experiencia)
 → descartado (si es ruido)

INSTRUCCIÓN PARA LA IA

Al iniciar cualquier tarea:

1. Lee STM → contexto inmediato
2. Busca en LTM → conocimiento relevante
3. Busca en episodic → experiencias similares
4. Aplica CoT.md para razonar
5. Al terminar → actualiza memoria correspondiente



.ai/memory/STM.md — Memoria a Corto Plazo

Markdown

```
# STM — Memoria a Corto Plazo
> Se limpia al inicio de cada sesión nueva.
> Solo contiene el contexto de la sesión ACTUAL.

---

## Sesión actual
- **Fecha:** [AUTO: fecha inicio]
- **Objetivo:** [qué se quiere lograr hoy]
- **Estado:** [en progreso / bloqueado / completado]

## Tareas en progreso
- [ ] Tarea 1 → archivo: `src/...` → estado: ...
- [ ] Tarea 2 → archivo: `src/...` → estado: ...

## Decisiones tomadas esta sesión
- [DECISIÓN]: Usar X en vez de Y → porque Z
- [DECISIÓN]: No implementar X todavía → bloqueado por Y

## Variables/contexto activo
- Rama git: `feature/xxx`
- Último archivo editado: `src/...`
- Error activo: ninguno / [descripción]

## Notas rápidas
- ...

---
> ⚠ AL CERRAR SESIÓN: mover lo relevante a LTM.md o episodic.md
```



.ai/memory/LTM.md — Memoria a Largo Plazo

Markdown

```
# LTM — Memoria a Largo Plazo
> Conocimiento técnico acumulado entre sesiones y proyectos.
> NUNCA se limpia. Solo se actualiza y mejora.

---

## Stack y tecnologías

### [Tecnología: ej. Prisma + PostgreSQL]
- **Aprendido:** 2024-01-15
- **Confianza:** ● Alta
- **Conocimiento:**
  - Las migraciones deben hacerse con `prisma migrate dev` en local
  - En producción usar `prisma migrate deploy` NUNCA `dev`
  - Los tipos se generan automáticamente con `prisma generate`
- **Fuente:** experiencia propia + error cometido el 2024-01-15
- **Tags:** #prisma #postgresql #migraciones #produccion
```

```
---

### [Patrón: Manejo de errores en Express]
- **Aprendido:** 2024-01-20
- **Confianza:** ● Alta
- **Conocimiento:**
  - Siempre usar middleware de error centralizado en `app.ts`
  - Los errores async necesitan `next(error)` o `express-async-errors`
  - Formato estándar: `{ error: string, code: string, details?: any }`
- **Tags:** #express #errores #middleware #async

---

## Patrones de arquitectura probados
| Patrón | Resultado | Proyecto | Fecha |
|-----|-----|-----|-----|
| Repository Pattern | ✔ Exitoso | proyecto-a | 2024-01 |
| God Service | ✖ Problemático | proyecto-b | 2024-02 |
| CQRS simplificado | ✔ Exitoso | proyecto-c | 2024-03 |

---

## Reglas aprendidas (no en rules.md porque son específicas)
- Si el proyecto usa Prisma, SIEMPRE revisar que `schema.prisma` esté sincronizado antes de codear
- Los JWT sin refresh token causan problemas de UX — siempre implementar ambos
- Zod es preferible a Joi para proyectos TypeScript (tipos nativos)

---

## Anti-patrones identificados
- ✖ Usar `any` en TypeScript termina en deuda técnica masiva
- ✖ Migrations con datos en prod sin backup previo → desastre x2
- ✖ No validar inputs en el router → vulnerabilidades SQL/XSS

---

> 📅 Última actualización: [fecha]
> 📊 Total de entradas: [n]
```

.ai/memory/episodic.md — Memoria Episódica

Markdown

```
# Memoria Episódica
> Historial de proyectos, tareas y experiencias concretas.
> Responde a: "¿Ya hice algo así antes? ¿Cómo me fue?"

---
```

```
## Episodio: [nombre descriptivo]
- **ID:** EP-001
- **Fecha:** 2024-01-15
- **Proyecto:** mi-proyecto-a
- **Tarea:** Implementar autenticación JWT con refresh tokens
- **Resultado:** ✔ Exitoso
- **Tiempo real:** 4h (estimado: 2h)
```

- **Confianza:** ● Alta

Qué hice

1. Instalar `jsonwebtoken` + `@types/jsonwebtoken`
2. Crear `src/middleware/auth.ts` con verificación
3. Crear ruta `/auth/refresh` con validación de refresh token en Redis
4. Tests con Jest + supertest

Qué salió mal

- Olvidé manejar el caso de token expirado vs token inválido (son errores distintos)
- No consideré la invalidación de tokens al cambiar password

Qué aprendí

- Separar errores: `TOKEN\_EXPIRED` (renovar) vs `TOKEN\_INVALID` (logout)
- Guardar refresh tokens en Redis con TTL = tiempo de expiración

Archivos clave creados

- `src/middleware/auth.ts`
- `src/routes/auth.ts`
- `src/services/tokenService.ts`

Tags

#auth #jwt #redis #tokens #middleware

Episodio: [otro episodio]

- **ID:** EP-002

...

Búsqueda rápida por tags

Tag	Episodios relacionados
#auth	EP-001
#prisma	EP-003, EP-007
#docker	EP-005



.ai/memory/index.md — Índice Maestro

Markdown

Índice de Memoria — Búsqueda Rápida

Por tecnología

Tech	LTM	Episodios	Errores
TypeScript	LTM#ts	EP-001, EP-003	ERR-002
Prisma	LTM#prisma	EP-003, EP-007	ERR-001, ERR-005
Express	LTM#express	EP-001, EP-002	ERR-003
Docker	LTM#docker	EP-005	-
Redis	LTM#redis	EP-001, EP-006	ERR-004

Por tipo de problema

Problema	Dónde buscar	Confianza
Autenticación JWT	EP-001, LTM#auth	● Alta

Migraciones Prisma	LTM#prisma, ERR-001	● Alta
Manejo de errores Express	LTM#express, PAT-002	● Alta
Docker + Node en prod	EP-005, LTM#docker	● Media

Estadísticas de memoria

- Total episodios: 0
- Total entradas LTM: 0
- Total errores registrados: 0
- Total patrones: 0
- Última actualización: -

.ai/reasoning/CoT.md — Chain of Thought

Markdown

Chain of Thought – Protocolo de Razonamiento

INSTRUCCIÓN OBLIGATORIA

Antes de responder cualquier tarea NO trivial, la IA DEBE seguir este proceso:

Plantilla CoT

[FASE 1 - ENTENDER]

- ¿Qué me están pidiendo exactamente?
- ¿Cuáles son los criterios de éxito?
- ¿Qué NO me están pidiendo (límites)?

[FASE 2 - RECUPERAR MEMORIA]

- ¿Existe algo en LTM relevante? → buscar en LTM.md
- ¿Existe un episodio similar? → buscar en episodic.md
- ¿Hay errores conocidos sobre esto? → buscar en errors.md
- ¿Hay patrones aplicables? → buscar en patterns.md

[FASE 3 - PLANIFICAR]

- Paso 1: ...
- Paso 2: ...
- Paso N: ...
- Riesgos identificados: ...
- Alternativas consideradas: ...

- Decisión tomada: ... porque ...

[FASE 4 - EJECUTAR]

- Ejecutar el plan paso a paso
- Si aparece algo inesperado → volver a FASE 3

[FASE 5 - VERIFICAR]

- ¿El resultado cumple los criterios de éxito?
- ¿Se violó alguna regla de rules.md?
- ¿Hay algo que debería guardarse en memoria?

[FASE 6 - ACTUALIZAR MEMORIA]

- Si aprendí algo nuevo → LTM.md
- Si completé una tarea significativa → episodic.md
- Si cometí un error → errors.md
- Si encontré un patrón útil → patterns.md

text

Cuándo aplicar CoT completo vs simplificado

Tipo de tarea	CoT requerido
Nueva feature compleja	✅ Completo (6 fases)
Bug fix crítico	✅ Completo (6 fases)
Refactor de arquitectura	✅ Completo (6 fases)
Fix simple / typo	⚡ Simplificado (1,3,5)
Pregunta de consulta	⚡ Solo FASE 2 + respuesta
Documentación	⚡ Solo FASE 1 + ejecución

Regla de confianza

- 🟢 **Alta (>0.8):** Proceder con el plan
- 🟡 **Media (0.5-0.8):** Proceder pero notificar incertidumbre
- 🔴 **Baja (<0.5):** Pedir confirmación antes de ejecutar

Markdown

FactChecker — Verificación de Hechos

Propósito

Evitar que la IA afirme cosas incorrectas sobre el proyecto, el stack o las decisiones.

Antes de afirmar algo como VERDAD, verificar:

Nivel 1 — Verificar en memoria local

- ¿Está documentado en LTM.md? → score +0.3
- ¿Hay un episodio que lo confirme? → score +0.3
- ¿Está en context.md o rules.md? → score +0.4

Nivel 2 — Verificar en código real

- ¿Puedo leer el archivo que menciono? → confirmar antes de afirmar
- ¿La dependencia que menciono está en package.json? → verificar
- ¿La ruta que menciono existe en src/? → verificar

Nivel 3 — Incertidumbre

Si no puedo verificar → usar lenguaje de incertidumbre:

- ❌ "El archivo está en src/routes/auth.ts"
- ✅ "Creo que el archivo está en src/routes/auth.ts — ¿puedo verificarlo?"

Score de confianza por tipo de afirmación

Tipo	Score base	Aumenta si...
-----	-----	-----
Hechos del código (rutas, funciones)	0.3	Leí el archivo: +0.6
Conocimiento técnico general	0.7	Está en LTM: +0.2
Decisiones de arquitectura	0.5	Está en decisions.md: +0.4
Estimaciones de tiempo	0.3	Hay episodio similar: +0.3

Gate de decisión

- Score ≥ 0.8 → Afirmar con seguridad
- Score 0.5-0.8 → Afirmar con advertencia: "⚠️ Confianza media — verificar"
- Score < 0.5 → No afirmar — preguntar o investigar primero



.ai/logs/errors.md — Registro de Errores

Markdown

Registro de Errores y Soluciones

> Cada error registrado aquí es experiencia que evita repetir el mismo error.

ERR-001

- **Fecha:** -
- **Proyecto:** -


```

- **Error:** [descripción del error]
- **Causa raíz:** [por qué ocurrió]
- **Síntomas:** [cómo se manifestó]
- **Solución aplicada:** [qué se hizo para resolverlo]
- **Tiempo perdido:** -
- **Cómo evitarlo:** [regla o checklist]
- **Tags:** #tag1 #tag2

```

Checklist anti-errores frecuentes

Antes de deployar:

```

- [ ] Variables de .env configuradas en el servidor
- [ ] Migraciones ejecutadas (`prisma migrate deploy`)
- [ ] Tests pasando (`npm test`)
- [ ] Build sin errores (`npm run build`)
- [ ] Logs de errores revisados

```

Antes de hacer una PR:

```

- [ ] Código revisado con linter
- [ ] Sin `console.log` de debug
- [ ] Types correctos (sin `any`)
- [ ] Tests unitarios agregados

```



.ai/logs/learnings.md — Aprendizajes

Markdown

Aprendizajes Acumulados

```

> Todo lo que la IA aprende en cada proyecto.
> Este archivo hace que cada proyecto siguiente sea mejor.

```

Formato de aprendizaje

LEARN-001

```

- **Fecha:** -
- **Categoría:** arquitectura / performance / seguridad / dx / testing
- **Aprendizaje:** [qué se aprendió]
- **Contexto:** [en qué situación]
- **Aplicar cuando:** [condición para usar este aprendizaje]
- **No aplicar cuando:** [excepciones]
- **Confianza:** ●/●/●
- **Tags:** #tag1

```

Aprendizajes de alto impacto (Top 10)

> Los más importantes van aquí para lectura rápida

```

1. [Aprendizaje crítico 1]
2. [Aprendizaje crítico 2]
...

```



Actualización de `opencode.json`

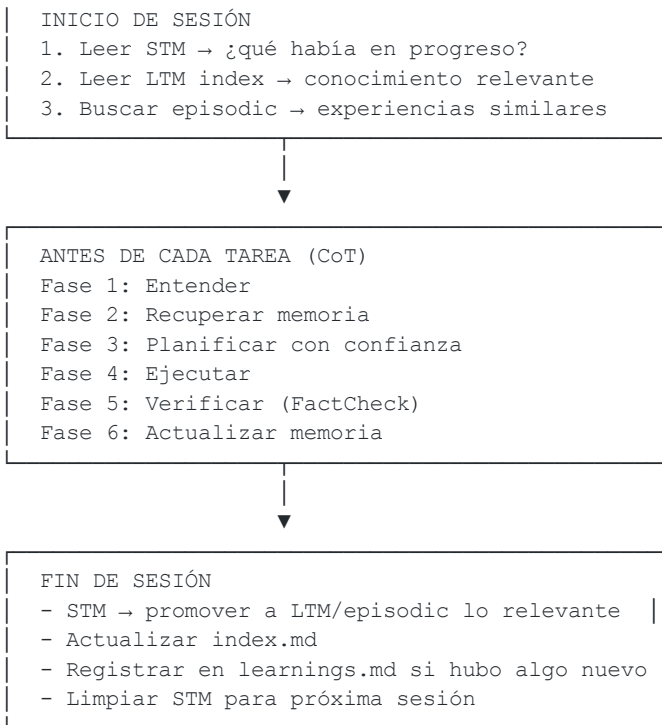
JSON

```
{
  "ai": {
    "path": ".ai",
    "memory": {
      "stm": ".ai/memory/STM.md",
      "ltm": ".ai/memory/LTM.md",
      "episodic": ".ai/memory/episodic.md",
      "index": ".ai/memory/index.md"
    },
    "reasoning": {
      "cot": ".ai/reasoning/CoT.md",
      "factcheck": ".ai/reasoning/factcheck.md",
      "patterns": ".ai/reasoning/patterns.md"
    },
    "logs": {
      "decisions": ".ai/logs/decisions.md",
      "errors": ".ai/logs/errors.md",
      "learnings": ".ai/logs/learnings.md"
    },
    "load_order": [
      "system.md",
      "rules.md",
      "context.md",
      "memory/index.md",
      "memory/LTM.md",
      "memory/episodic.md",
      "reasoning/CoT.md",
      "reasoning/factcheck.md",
      "agents.md"
    ]
  },
  "model": "gpt-4o",
  "tools": ["read", "write", "execute", "grep", "browser"],
  "temperature": 0.2
}
```



Flujo completo de trabajo con memoria

text



Resultado final

Con esta arquitectura tu proyecto:

Capacidad	Antes	Después
Recuerda sesiones anteriores	✗	✓ STM + episodic
Acumula conocimiento técnico	✗	✓ LTM

Evita repetir errores



✓ errors.md

Razona antes de actuar



✓ CoT

Verifica antes de afirmar



✓ FactCheck

Mejora con cada proyecto



✓ learnings.md

Busca experiencias similares



✓ episodic + index